Software Engineering
Engineering the Development of Software Systems

L05 – Software Engineering in Python – Exercises – Part 3

Marcello Missiroli and Giancarlo Succi
Dipartimento di Informatica – Scienza e Ingegneria
Università di Bologna

What is Pydantic?

- **Pydantic** is a Python library for:
  - Data validation
  - Data parsing
  - Settings and configuration management
- It is based on **Python type annotations** and uses them to:
  - Enforce type correctness at runtime
  - Automatically convert (coerce) input data into the correct types
  - Provide informative error messages when validation fails
- Pydantic combines the flexibility of dynamic typing with the safety of static typing.
- Widely used in real-world applications (APIs, configuration, data pipelines).

Pydantic setup

Exercise

1. Install the Pydantic extension via pip.

```
pip install pydantic
Optional email validation pip install "pydantic[email]"
Settings management (via .env files)
pip install pydantic-settings
```

2. Check that everything works: write a simple program that checks if Pydantic is correctly installed

Solution

```
try:
    import pydantic
    print("Pydantic is installed correctly.")
    print("Version:", pydantic.__version__)
except ImportError:
    print("Pydantic is NOT installed. Please run:")
    print("    pip install pydantic")
```

First Pydantic Model

Task

- Create a `Product` model with the following fields:
  - `name`: string
  - `price`: float
  - `in_stock`: boolean
- Instantiate the model using values of the wrong type (e.g., `price="10"` or `in_stock="true"`).
- Observe what happens:
  - When Pydantic automatically converts types
  - When Pydantic raises a `ValidationError`

Learning Points

`BaseModel`, type parsing, `ValidationError`.

First model – Possible Solution

Code

```
from pydantic import BaseModel, ValidationError

class Product(BaseModel):
    name:  str
    price:  float
    in_stock:  bool

# Valid instantiation
p1 = Product(name="Laptop", price=999.99, in_stock=True)
print(p1)

# Instantiation with convertible types
p2 = Product(name="Mouse", price="10", in_stock="true")
print(p2)

# Invalid types producing ValidationError
try:
    p3 = Product(name=123, price="ten", in_stock="yes")
except ValidationError as e:
    print("Validation error:")
```

Pydantic – Exercise 2

Exercise

- Extend the `Product` model by adding constraints:
  - `name` must have at least 2 characters
  - `price` must be greater than 0
- Use `Field()` to set:
  - `min_length`
  - `gt`
- Instantiate the model with invalid values to verify the errors.

Learning Points

`Field`, `min_length`, `gt`, multiple validation errors.

Extended model – Possible Solution

Code

```
from pydantic import BaseModel, Field, ValidationError

class Product(BaseModel):
    name:  str = Field(min_length=2)
    price:  float = Field(gt=0)
    in_stock:  bool

# Valid instance
p1 = Product(name="PC", price=1200, in_stock=True)
print(p1)

# Invalid instances to trigger errors
try:
    p2 = Product(name="A", price=-5, in_stock=True)
except ValidationError as e:
    print(e)
```

Output

```
2 validation errors for Product ...
```

Nested Data and JSON

Exercise

- Create a `Category` model with:
  `name`: string
  `department`: a string allowed only as "HR" or "IT" (use an Enum)
- Modify `Product` by adding the field:
  `category`: `Category`
- Validate the following JSON using `model_validate_json()`:

```
{ "name": "Laptop",
  "price": "1499",
  "in_stock": 1,
  "category": {
    "name": "Workstations",
    "department": "IT" }
```

- Finally, use `model_dump_json()` to return the result.

Learning Points

Nested models, Enum, `model_validate_json`, `model_dump_json`.

Possible solution

Code

```
from enum import Enum
from pydantic import BaseModel, ValidationError
import json
class Department(str, Enum):
  HR = "HR"
  IT = "IT"
class Category(BaseModel):
  name:  str
  department:  Department
class Product(BaseModel):
  name:  str
  price:  float
  instock:  bool
  category:  Category
jsondata = { "name":  "Laptop", "price":  "1499", "instock":  1, "category":
  {"name":  "Workstations", "department":  "IT" } }
product = Product.model_validate_json(json.dumps(jsondata))
print(product); print(product.model_dump_json())
```

n

Pydantic field evaluators

Exercise

- Add a new field to `Product`:
  - `discount`: float between 0 and 100 (percentage)
- Use `@field_validator("discount")` to ensure that:
  - The discounted price never falls below €1
- Test with valid and invalid values

Learning Points

`@field_validator`, custom validation logic, accessing other field values.

Possible solution

Code

```
from pydantic import BaseModel, Field, field_validator, ValidationError

class Product(BaseModel):
    name: str
    price: float
    in_stock: bool
    discount: float = Field(ge=0, le=100)

    @field_validator("discount")
    @classmethod
    def check_discount(cls, value):
        # Range check
        if not 0 <= value <= 100:
            raise ValueError("Discount must be between 0 and 100 percent")
        return value
# Valid example
p1 = Product(name="Laptop", price=1000, in_stock=True, discount=10); print(p1)
# Invalid example
p2 = Product(name="Mouse", price=5, in_stock=True, discount=150); print(p2)
```

Model validator

Exercise

- Add the following business rule:
  - If `category.department == "IT"`, then `discount` must not exceed 10%.
- Implement it with: `@model_validator(mode="after")`.
- Test these cases:
  - IT product with `discount=5` → OK
  - IT product with `discount=50` → error
  - HR product with `discount=50` → OK

Learning Points

`@model_validator`, cross-field validation, business rules.

Possible Solution (1/4)

Code

```
from enum import Enum
from typing import Self
from pydantic import (
    BaseModel, Field,
    ValidationError, model_validator
)

class Department(str, Enum):
    HR = "HR"
    IT = "IT"

class Category(BaseModel):
    name: str
    department: Department
```

Possible Solution (2/4)

Code

```
class Product(BaseModel):
    name: str
    price: float
    in_stock: bool
    discount: float = Field(ge=0, le=100)
    category: Category

    @model_validator(mode="after")
    def check_it_discount(self) -> Self:
        if self.category.department == Department.IT and self.discount > 10:
            raise ValueError(
                "IT products cannot have a discount higher than 10%."
            )
        return self
```

Possible Solution (3/4)

Code

```
product_it_ok = Product(
    name="Laptop",
    price=1000,
    in_stock=True,
    discount=5,
    category=Category(name="Workstations", department="IT"),
)
print("IT OK:", product_it_ok)
try:
    product_it_bad = Product(
        name="Server",
        price=5000,
        in_stock=True,
        discount=50,
        category=Category(name="Infrastructure", department="IT"),
    )
except ValidationError as e:
    print("IT error:\n", e)
```

Possible Solution (4/4)

Code

```
product_hr_ok = Product(
    name="HR Suite",
    price=2000,
    in_stock=True,
    discount=50,
    category=Category(name="Admin Tools", department="HR"),
)
print("HR OK:", product_hr_ok)
```

Questions?

End of the part three